

Labs 7 - Quantum Pattern Recognition

Mateusz Tabaszewski 151945

Introduction

This project explores the application of quantum algorithms in pattern recognition, specifically utilizing the SWAP test to quantify the similarity between two-qubit quantum states. In the context of Quantum Machine Learning (QML), the ability to determine the "closeness" or fidelity of different data patterns is fundamental for classification and clustering tasks. By implementing a circuit featuring Hadamard gates and controlled-SWAP (Fredkin) operations, we demonstrate how quantum interference can be used to extract the overlap between a test state $|\psi\rangle$ and a library of 16 distinct reference patterns $|\phi\rangle$, providing a practical look at state comparison on quantum hardware.

Imports

```
In [65]: import pandas as pd
from qiskit import QuantumCircuit
from qiskit.visualization import plot_distribution
from qiskit_aer import Aer
from qiskit.compiler import transpile
import numpy as np
```

Functions

```
In [66]: def swap_test_circuit(psi_prep_fn, phi_prep_fn):
    qc = QuantumCircuit(5, 1)
    psi_prep_fn(qc, [1, 2])
    phi_prep_fn(qc, [3, 4])
    qc.barrier()
    qc.h(0)
    qc.cswap(0, 1, 3)
    qc.cswap(0, 2, 4)
    qc.h(0)
    qc.measure(0, 0)
    return qc

def prepare_phi_plusplus(qc, qubits):
    qc.h(qubits[0])
    qc.h(qubits[1])
    return qc

def prepare_phi_minusminus(qc, qubits):
    pass

def prepare_phi_01(qc, qubits):
    qc.x(qubits[0])
    return qc

def prepare_phi_10(qc, qubits):
    qc.x(qubits[1])
    return qc

def prepare_phi_plus0(qc, qubits):
    qc.h(qubits[1])
    return qc

def prepare_phi_plus1(qc, qubits):
    qc.x(qubits[0])
    qc.h(qubits[1])
    return qc

def prepare_phi_1plus(qc, qubits):
    qc.h(qubits[0])
    qc.x(qubits[1])
    return qc

def prepare_phi_0plus(qc, qubits):
    qc.h(qubits[0])
    return qc

def prepare_psi_00(qc, qubits):
    return qc
```

```

def prepare_phi_plusminus(qc, qubits):
    qc.x(qubits[0])
    qc.h(qubits[0])
    qc.h(qubits[1])
    return qc

def prepare_phi_l1(qc, qubits):
    qc.x(qubits[0])
    qc.x(qubits[1])
    return qc

def prepare_phi_minusplus(qc, qubits):
    qc.h(qubits[0])
    qc.x(qubits[1])
    qc.h(qubits[1])
    return qc

def prepare_phi_0minus(qc, qubits):
    qc.x(qubits[0])
    qc.h(qubits[0])
    return qc

def prepare_phi_minus1(qc, qubits):
    qc.x(qubits[0])
    qc.x(qubits[1])
    qc.h(qubits[1])
    return qc

def prepare_phi_minus0(qc, qubits):
    qc.x(qubits[1])
    qc.h(qubits[1])
    return qc

def prepare_phi_lminus(qc, qubits):
    qc.x(qubits[0])
    qc.h(qubits[0])
    qc.x(qubits[1])
    qc.h(qubits[1])
    return qc

def calculate_results(counts):
    shots = sum(counts.values())
    p0 = counts.get('0', 0) / shots
    fidelity = 2 * p0 - 1
    overlap = np.sqrt(max(0, fidelity))
    return p0, fidelity, overlap

def display_lab_results(data_dict):
    df = pd.DataFrame(data_dict)
    df['P(0)'] = df['P(0)'].map('{:.5f}'.format)
    df['Fidelity'] = df['Fidelity'].map('{:.5f}'.format)
    df['Overlap'] = df['Overlap'].map('{:.5f}'.format)

    return df

```

Task 1 - Swap Tests

The PSI state $|00\rangle$

```

In [67]: results = {}
results["State"] = []
results["P(0)"] = []
results["Fidelity"] = []
results["Overlap"] = []

```

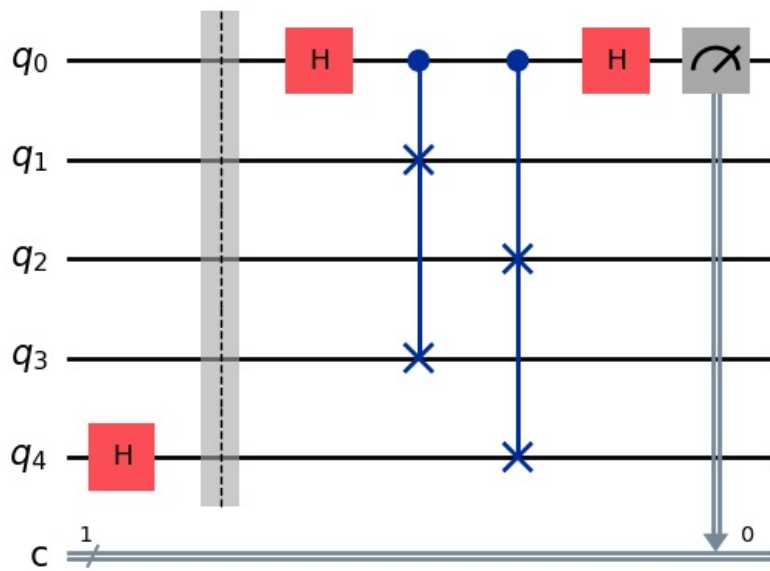
State Phi $|+0\rangle$

```

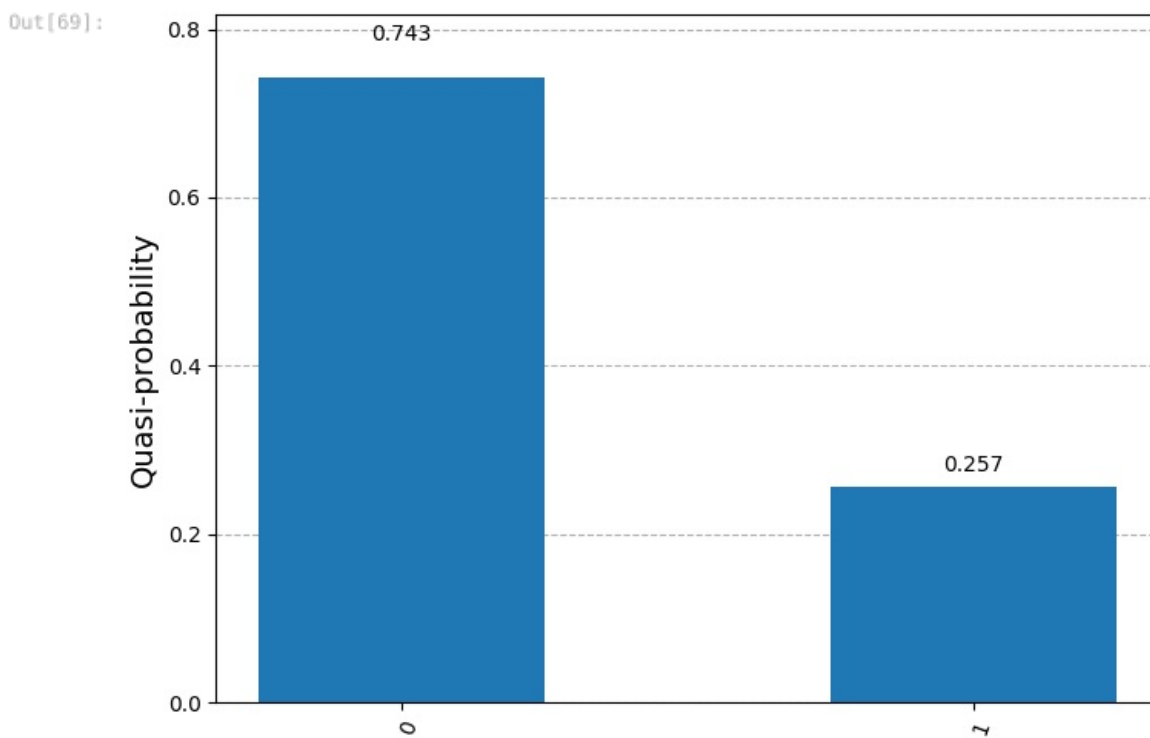
In [68]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_plus0)
qc.draw("mpl")

```

Out [68]:



```
In [69]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```



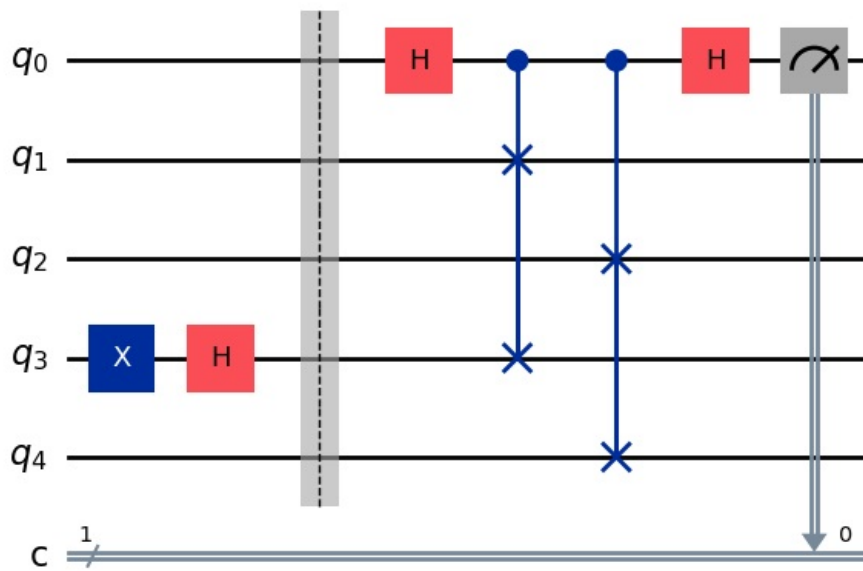
```
In [70]: p0, fid, over = calculate_results(counts)
results["State"].append("+0")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.74316, Fidelity: 0.48633, Overlap: 0.69737

State Phi $|0\rangle$ ->

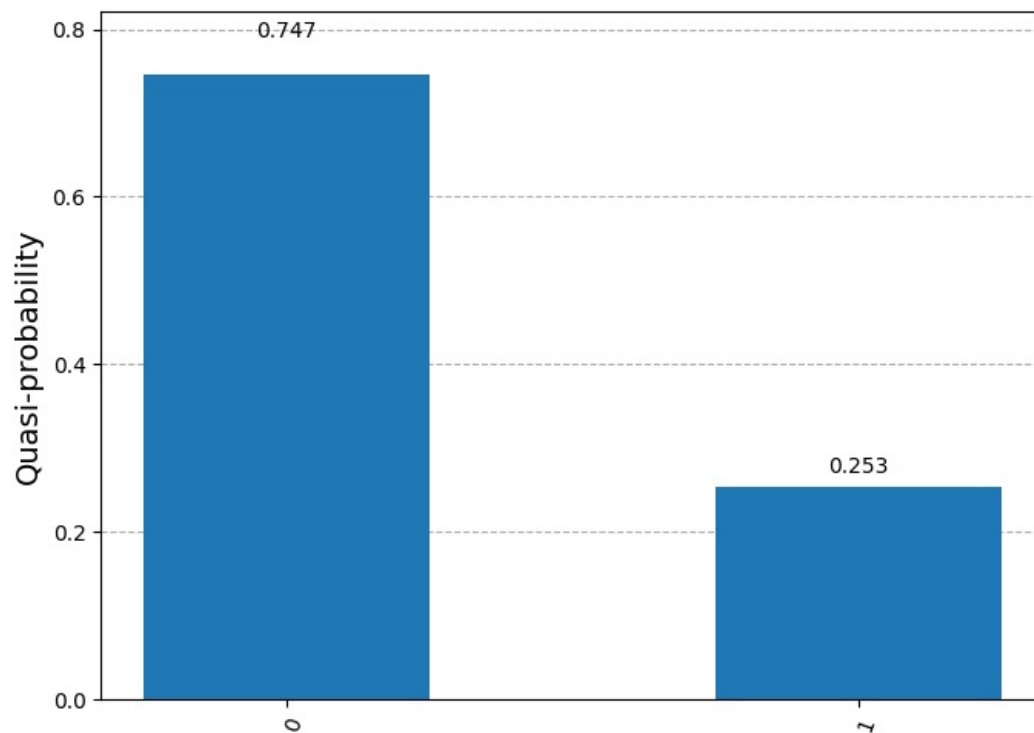
```
In [71]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_0minus)
qc.draw("mpl")
```

Out[71]:



```
In [72]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[72]:



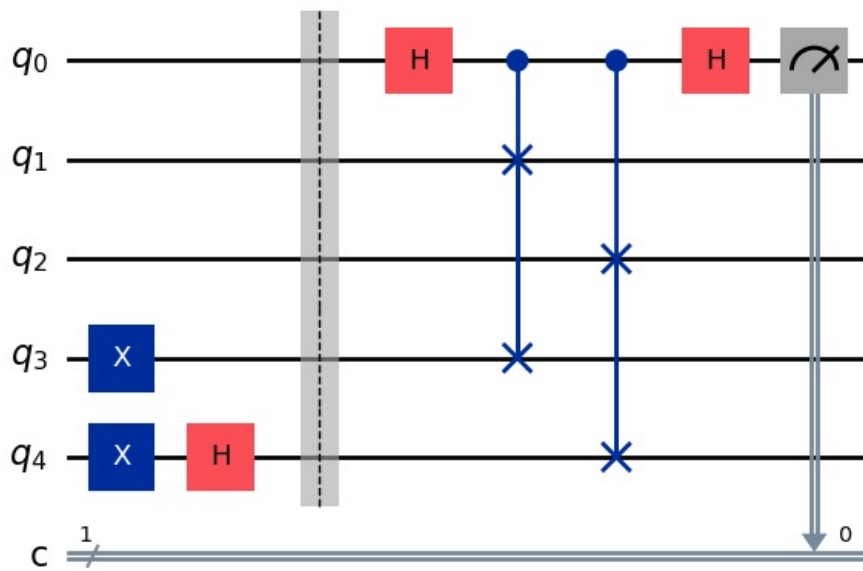
```
In [73]: p0, fid, over = calculate_results(counts)
results["State"].append("0-")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.74658, Fidelity: 0.49316, Overlap: 0.70226

State Φ $|-1\rangle$

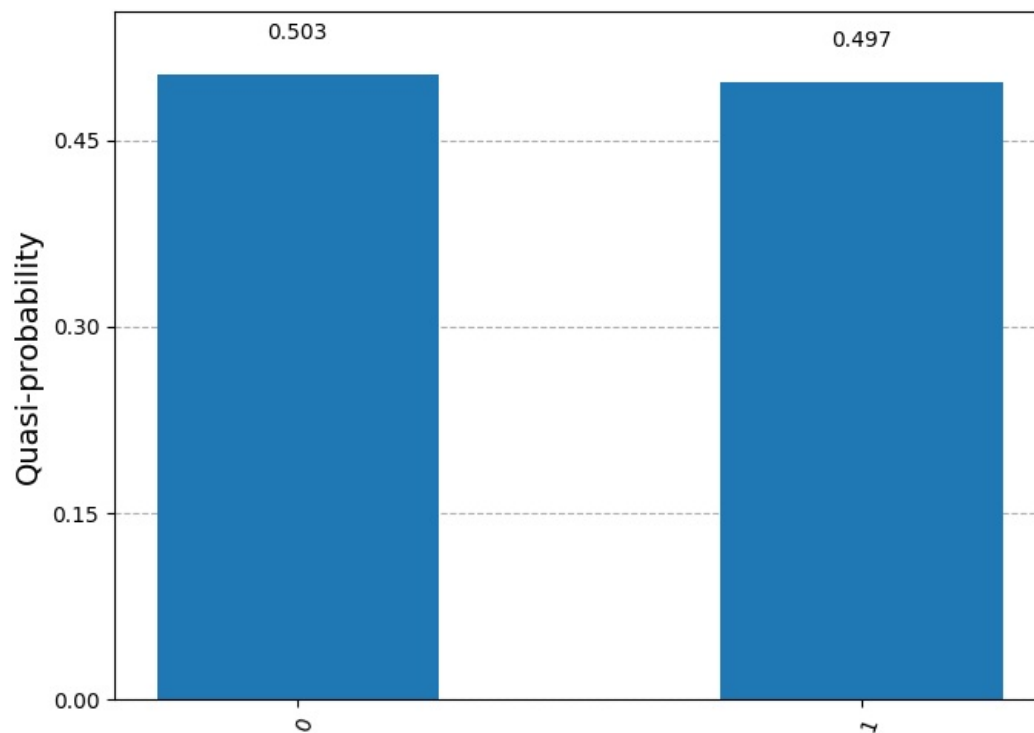
```
In [74]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_minus1)
qc.draw("mpl")
```

Out [74]:



```
In [75]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out [75]:



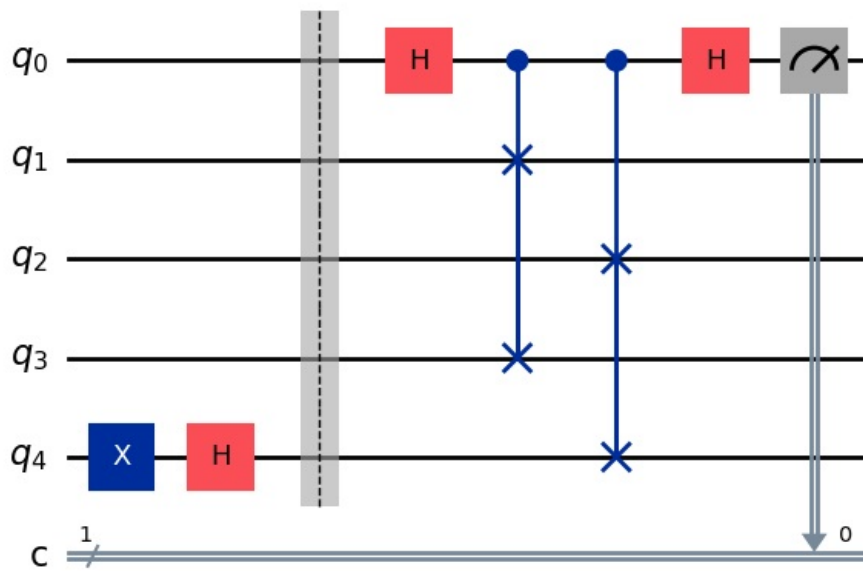
```
In [76]: p0, fid, over = calculate_results(counts)
results["State"].append("-1")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.50342, Fidelity: 0.00684, Overlap: 0.08268

State Φ $|-0\rangle$

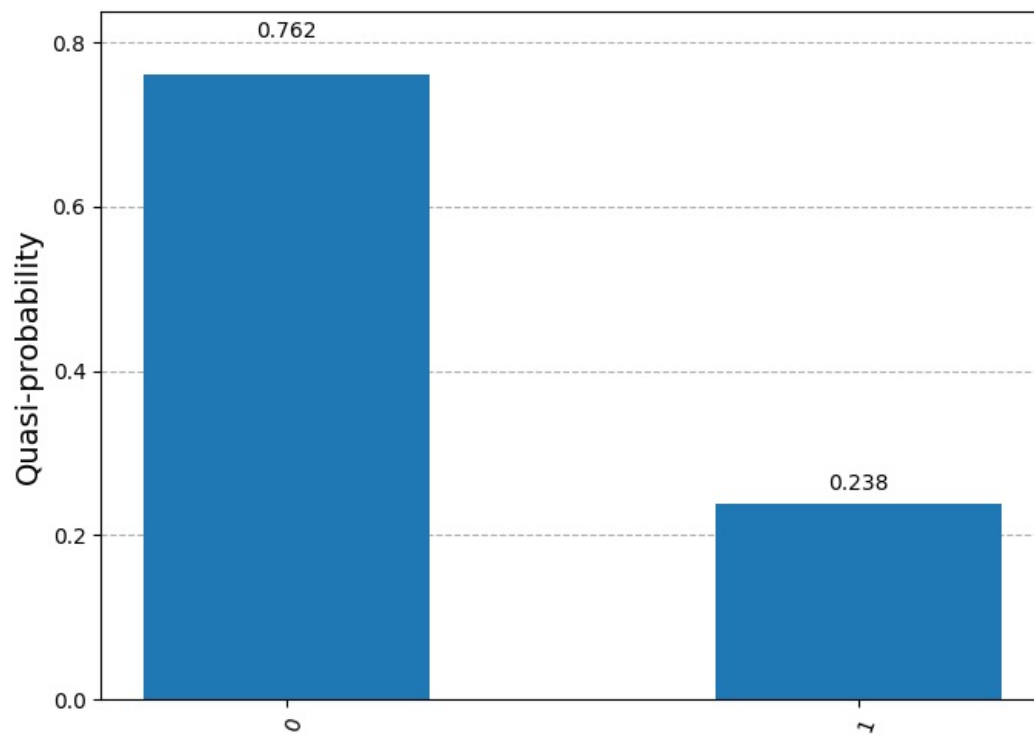
```
In [77]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_minus0)
qc.draw("mpl")
```

Out[77]:



```
In [78]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[78]:



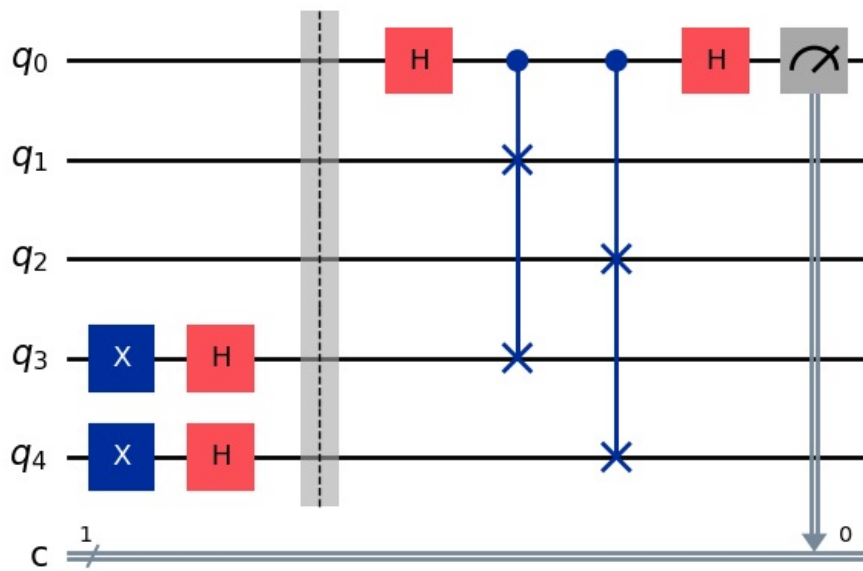
```
In [79]: p0, fid, over = calculate_results(counts)
results["State"].append("-0")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.76172, Fidelity: 0.52344, Overlap: 0.72349

State Φ $|1\rangle \rightarrow$

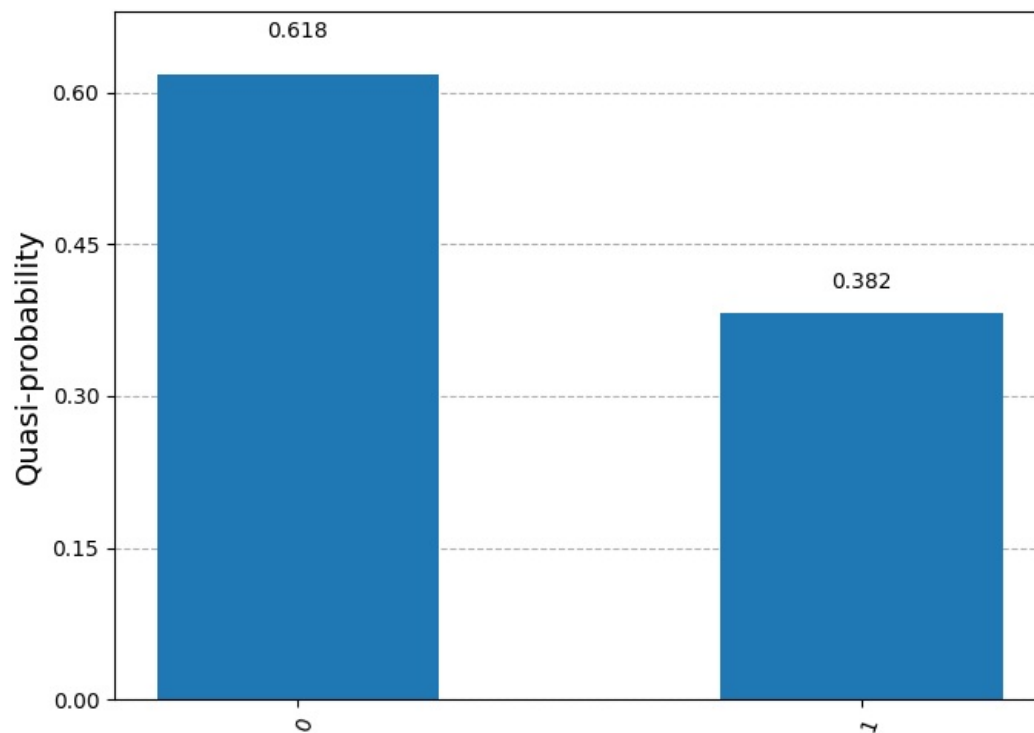
```
In [80]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_1minus)
qc.draw("mpl")
```

Out[80]:



```
In [81]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[81]:



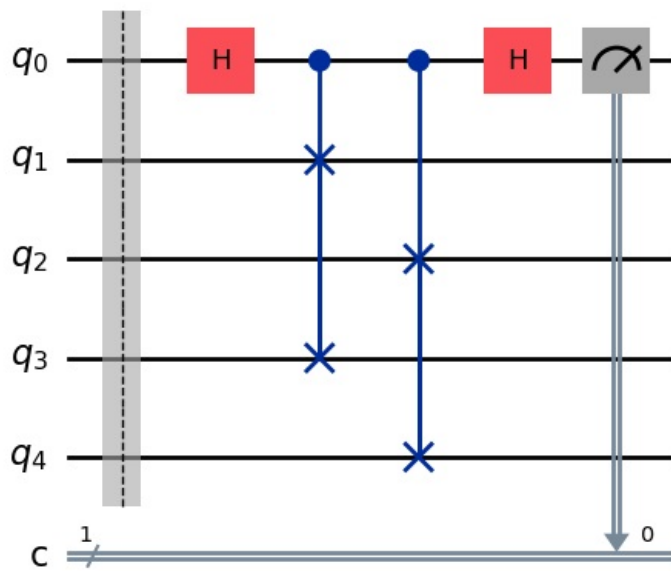
```
In [82]: p0, fid, over = calculate_results(counts)
results["State"].append("1-")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.61816, Fidelity: 0.23633, Overlap: 0.48614

State Φ $| \rightarrow$

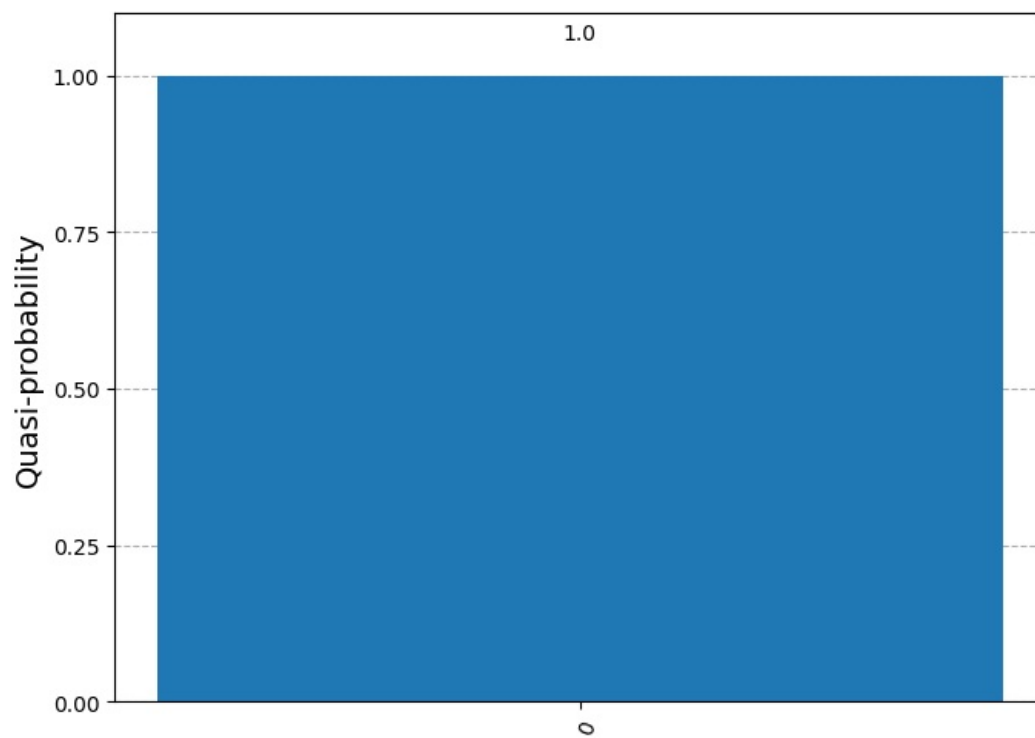
```
In [83]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_minusminus)
qc.draw("mpl")
```

Out[83]:



```
In [84]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[84]:



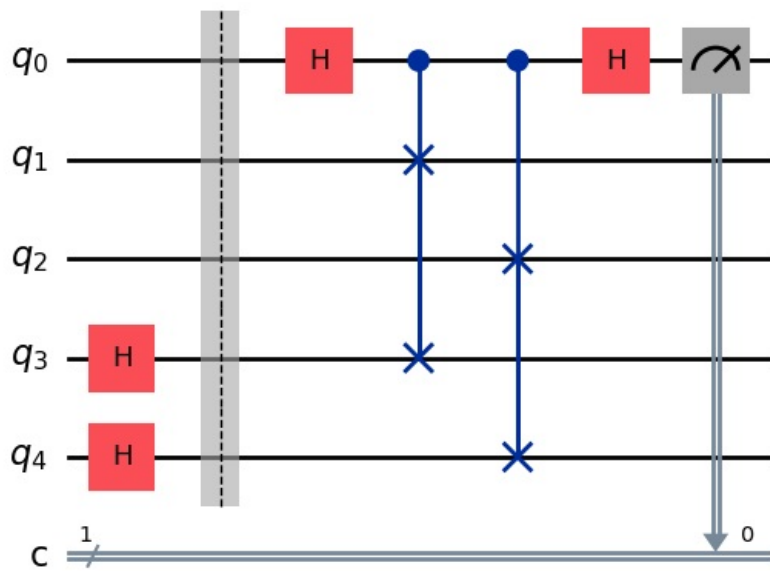
```
In [85]: p0, fid, over = calculate_results(counts)
results["State"].append("---")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 1.00000, Fidelity: 1.00000, Overlap: 1.00000

State Phi $|++\rangle$

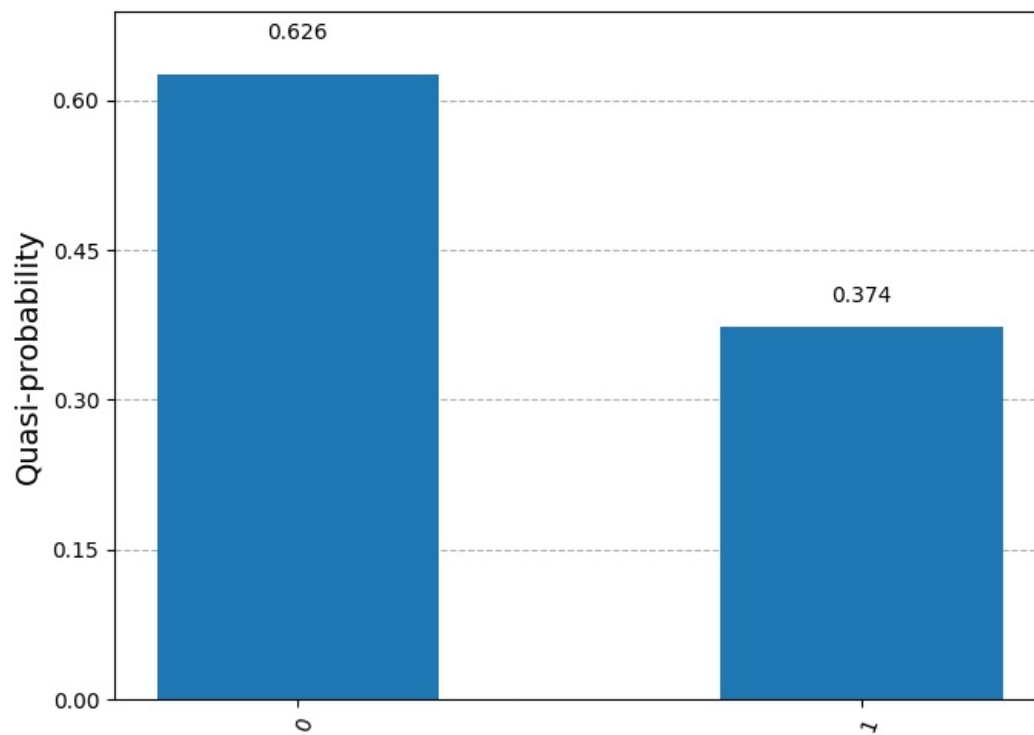
```
In [86]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_plusplus)
qc.draw("mpl")
```


Out[86]:



```
In [87]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[87]:



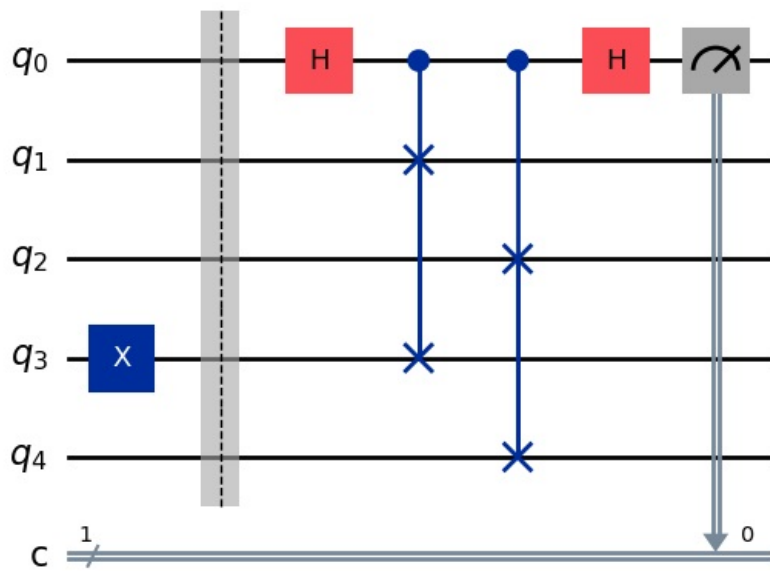
```
In [88]: p0, fid, over = calculate_results(counts)
results["State"].append("++")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.62646, Fidelity: 0.25293, Overlap: 0.50292

State Phi $|01\rangle$

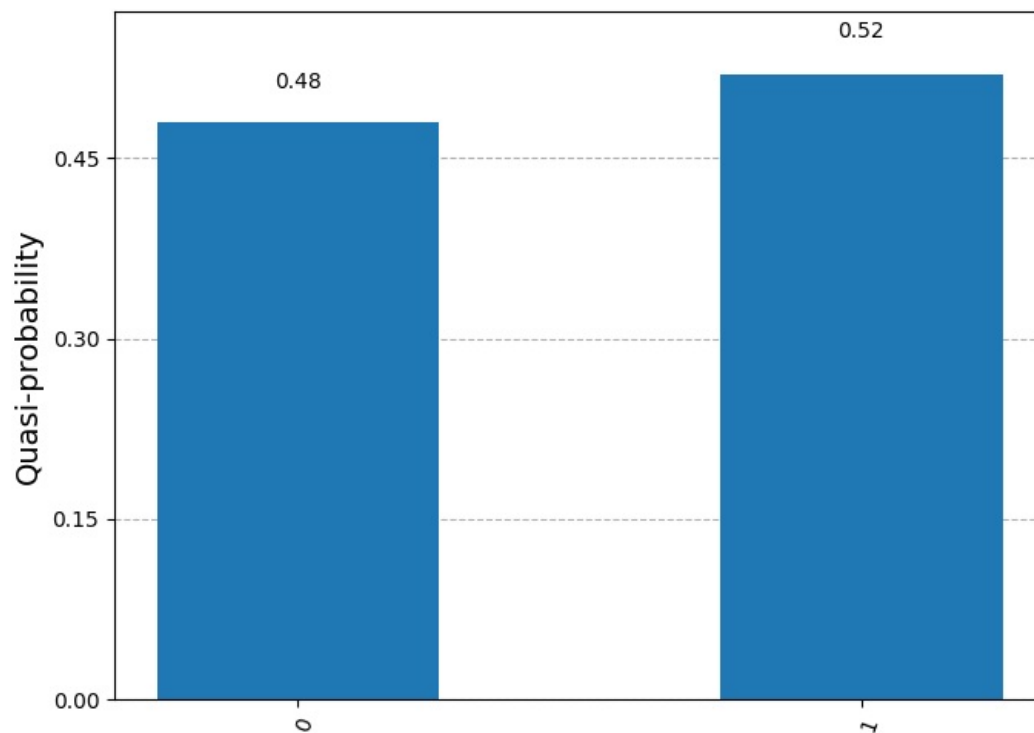
```
In [89]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_01)
qc.draw("mpl")
```

Out[89]:



```
In [90]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[90]:



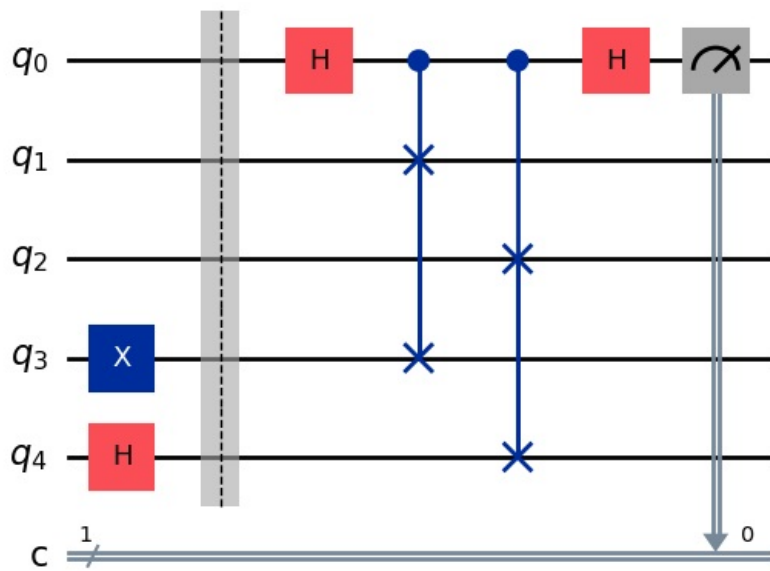
```
In [91]: p0, fid, over = calculate_results(counts)
results["State"].append("01")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.47998, Fidelity: -0.04004, Overlap: 0.00000

State Φ $|+1\rangle$

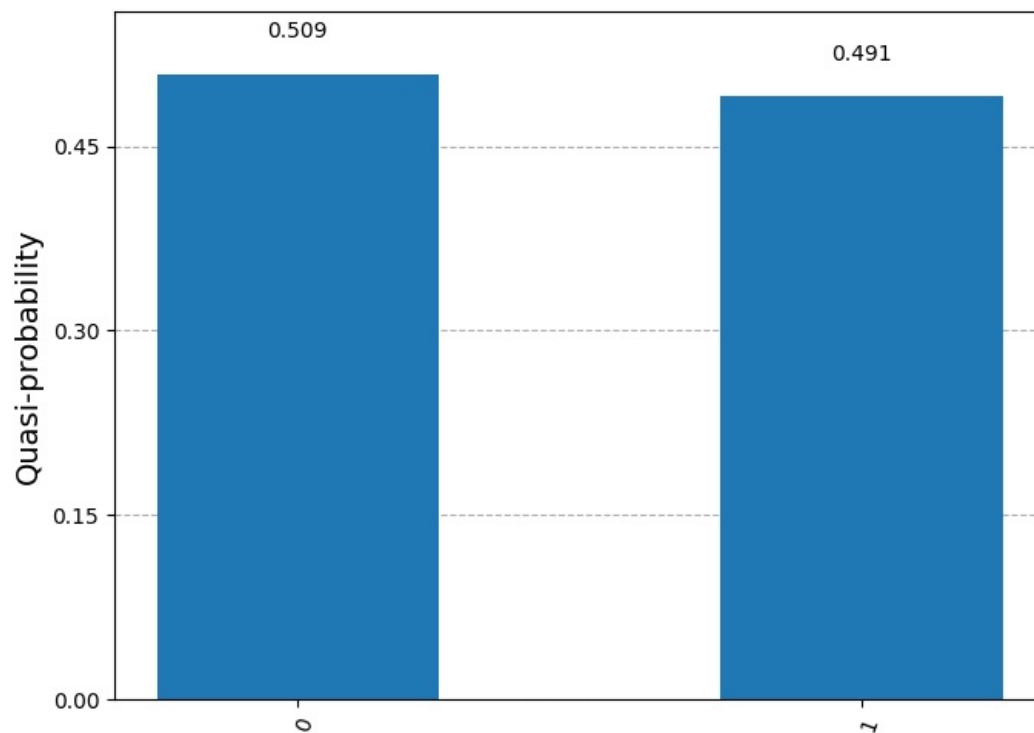
```
In [92]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_plus1)
qc.draw("mpl")
```

Out[92]:



```
In [93]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[93]:



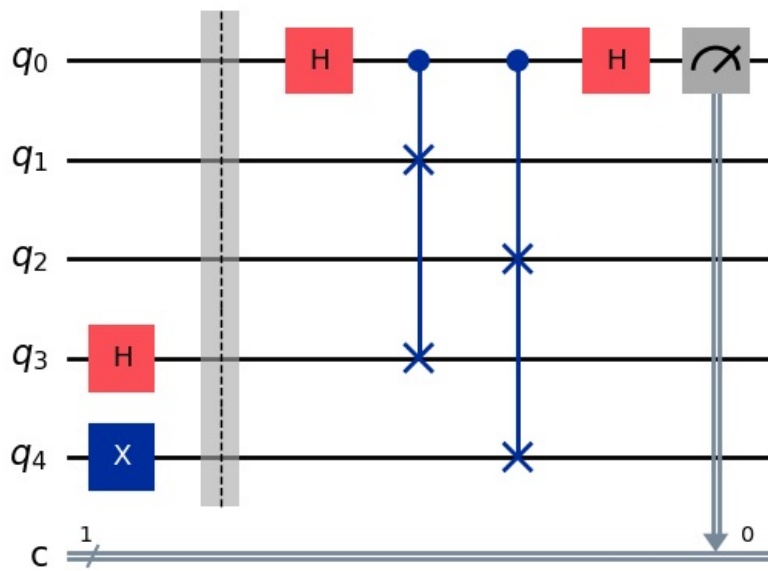
```
In [94]: p0, fid, over = calculate_results(counts)
results["State"].append("+1")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.50879, Fidelity: 0.01758, Overlap: 0.13258

State $\Phi |1+\rangle$

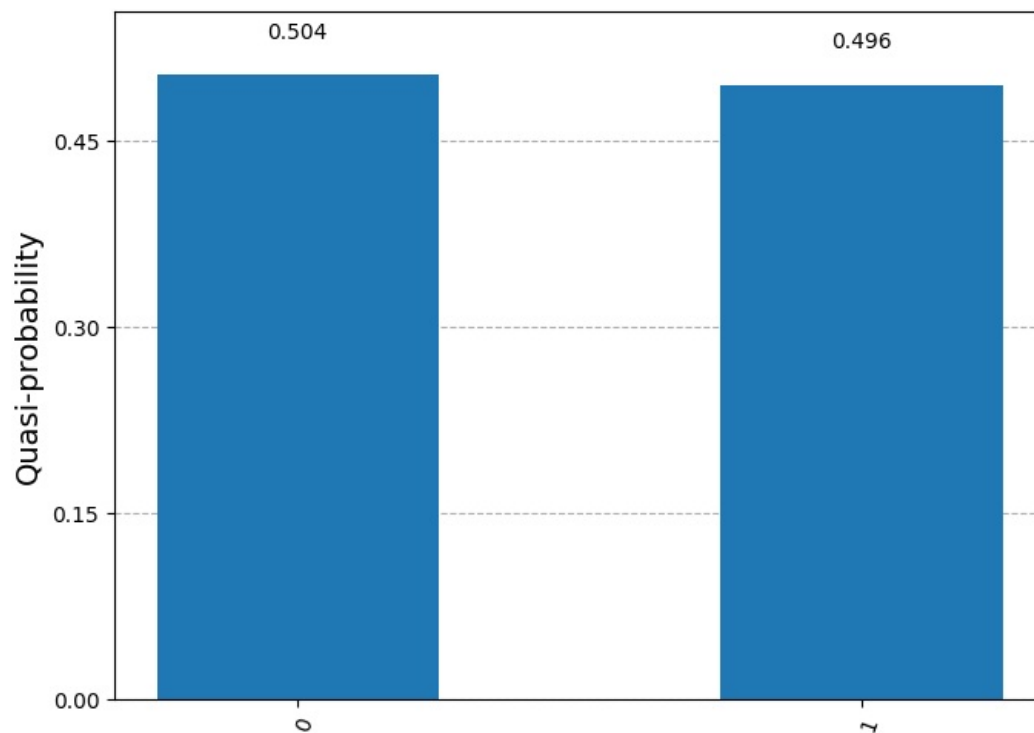
```
In [95]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_1plus)
qc.draw("mpl")
```

Out[95]:



```
In [96]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[96]:



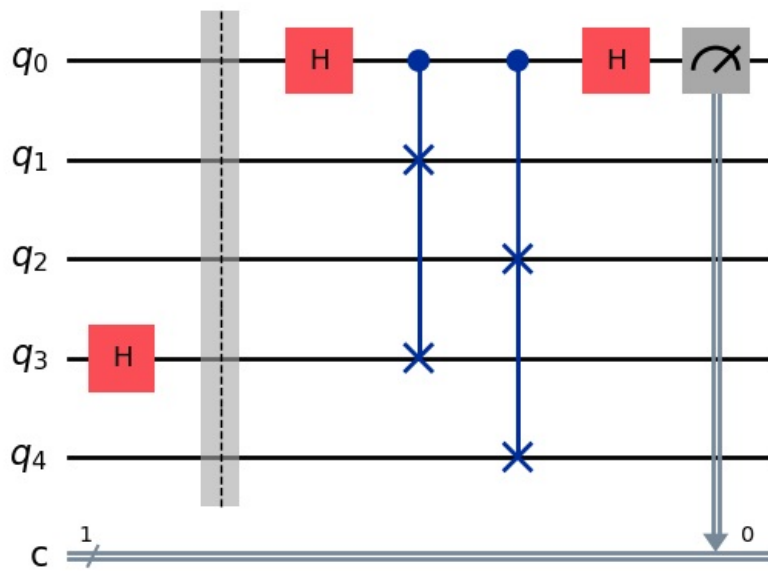
```
In [97]: p0, fid, over = calculate_results(counts)
results["State"].append("1+")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.50439, Fidelity: 0.00879, Overlap: 0.09375

State Φ $|0+\rangle$

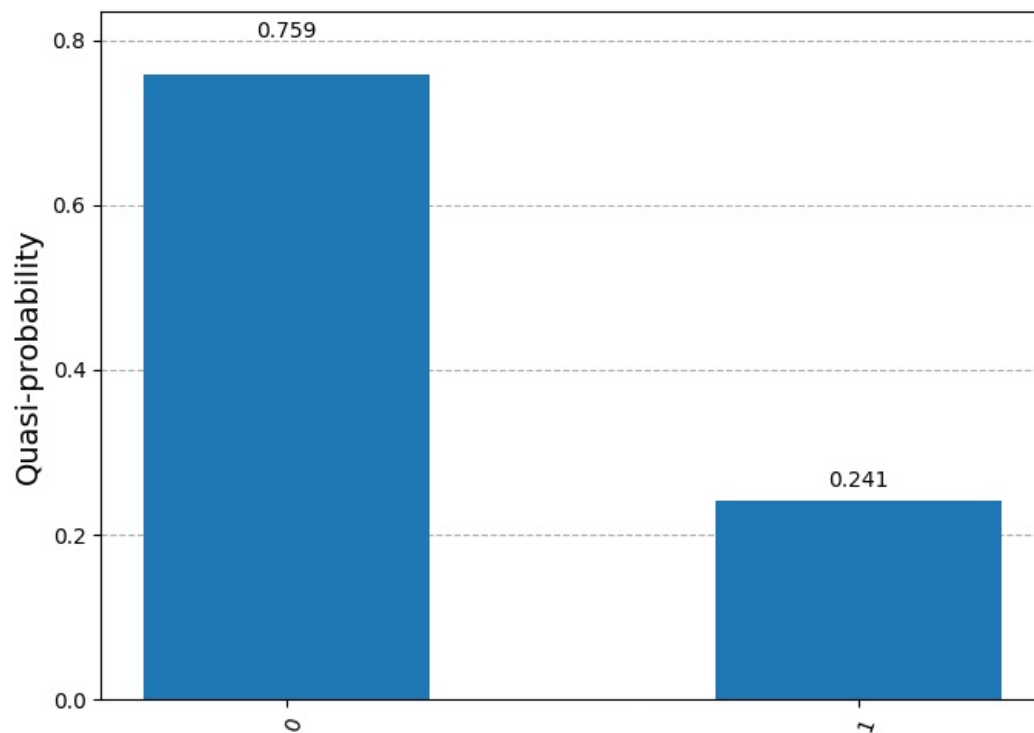
```
In [98]: qc = swap_test_circuit(prepare_psi_00, prepare_phi_0plus)
qc.draw("mpl")
```

Out[98]:



```
In [99]: backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[99]:



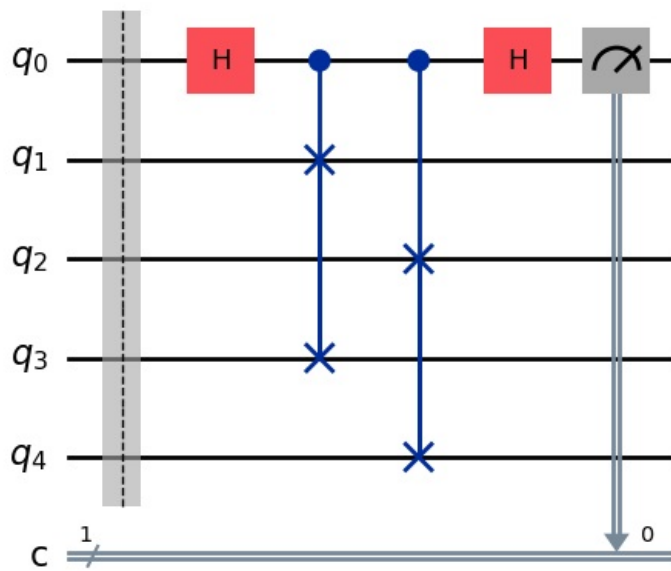
```
In [100]: p0, fid, over = calculate_results(counts)
results["State"].append("0+")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.75879, Fidelity: 0.51758, Overlap: 0.71943

State Φ $|00\rangle$

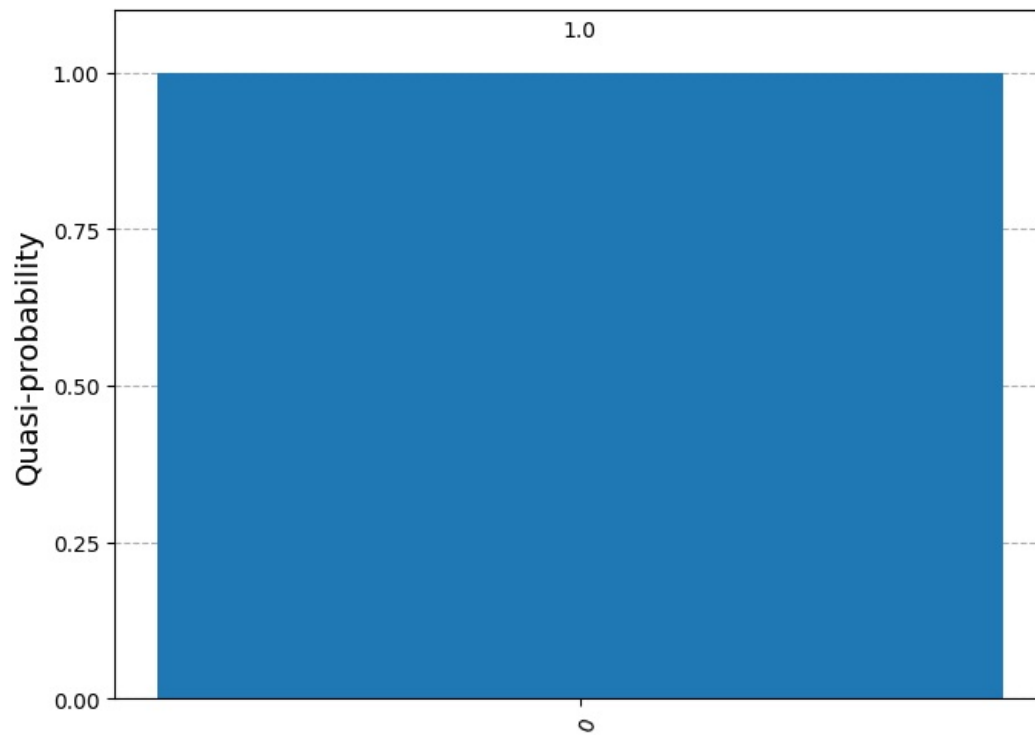
```
In [101]: qc = swap_test_circuit(prepare_psi_00, prepare_psi_00)
qc.draw("mpl")
```

Out[101...



```
In [102... backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[102...



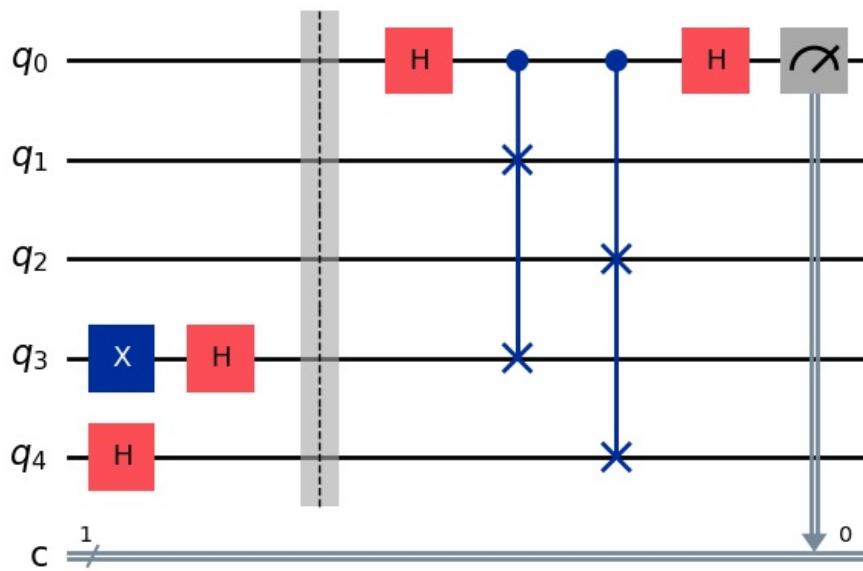
```
In [103... p0, fid, over = calculate_results(counts)
results["State"].append("00")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 1.00000, Fidelity: 1.00000, Overlap: 1.00000

State Φ $|+\rightarrow$

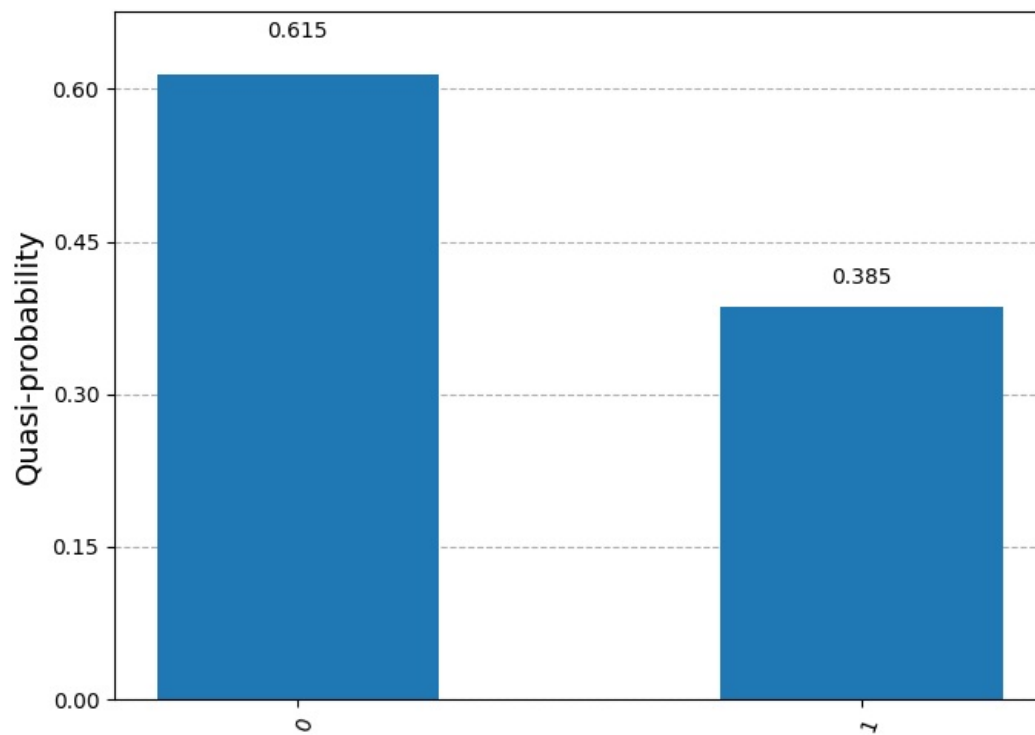
```
In [104... qc = swap_test_circuit(prepare_psi_00, prepare_phi_plusminus)
qc.draw("mpl")
```

Out [104...



```
In [105... backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out [105...



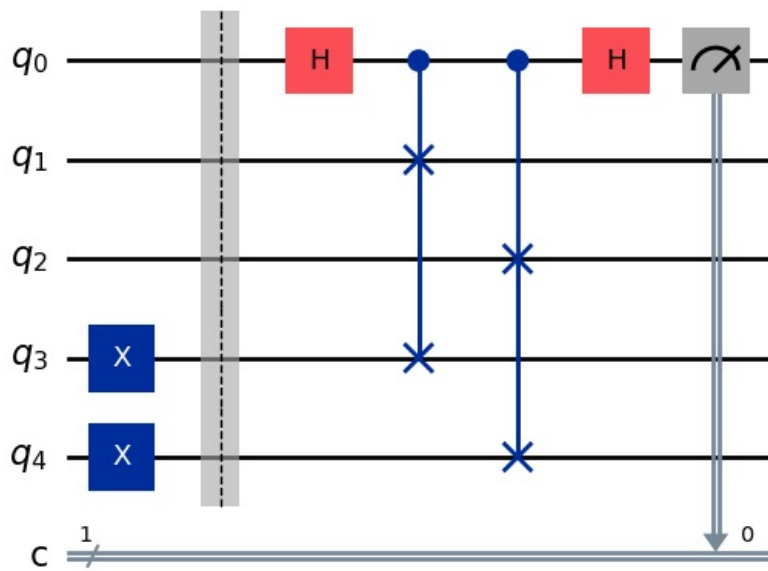
```
In [106... p0, fid, over = calculate_results(counts)
results["State"].append("+ -")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.61475, Fidelity: 0.22949, Overlap: 0.47905

State Φ $|11\rangle$

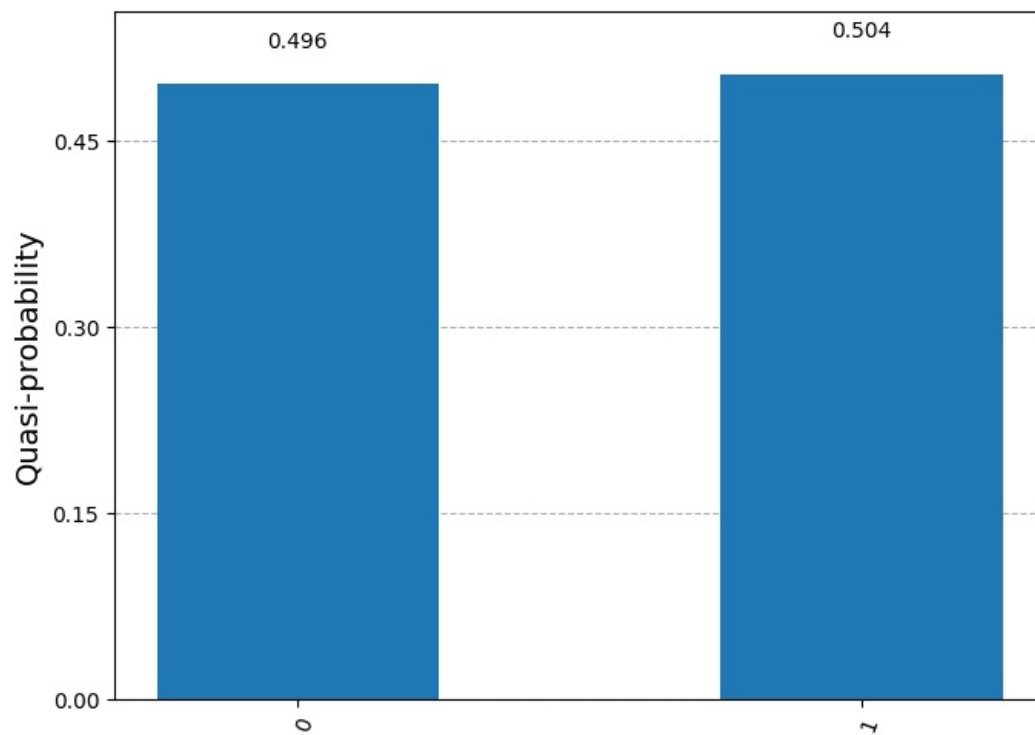
```
In [107... qc = swap_test_circuit(prepare_psi_00, prepare_phi_11)
qc.draw("mpl")
```

Out[107..



```
In [108.. backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[108..



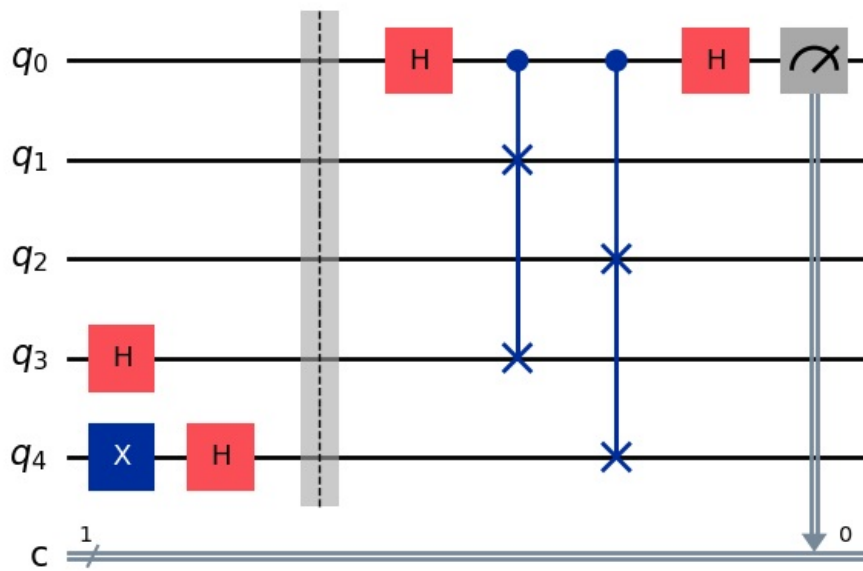
```
In [109.. p0, fid, over = calculate_results(counts)
results["State"].append("11")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.49609, Fidelity: -0.00781, Overlap: 0.00000

State Φ $|-\rangle$

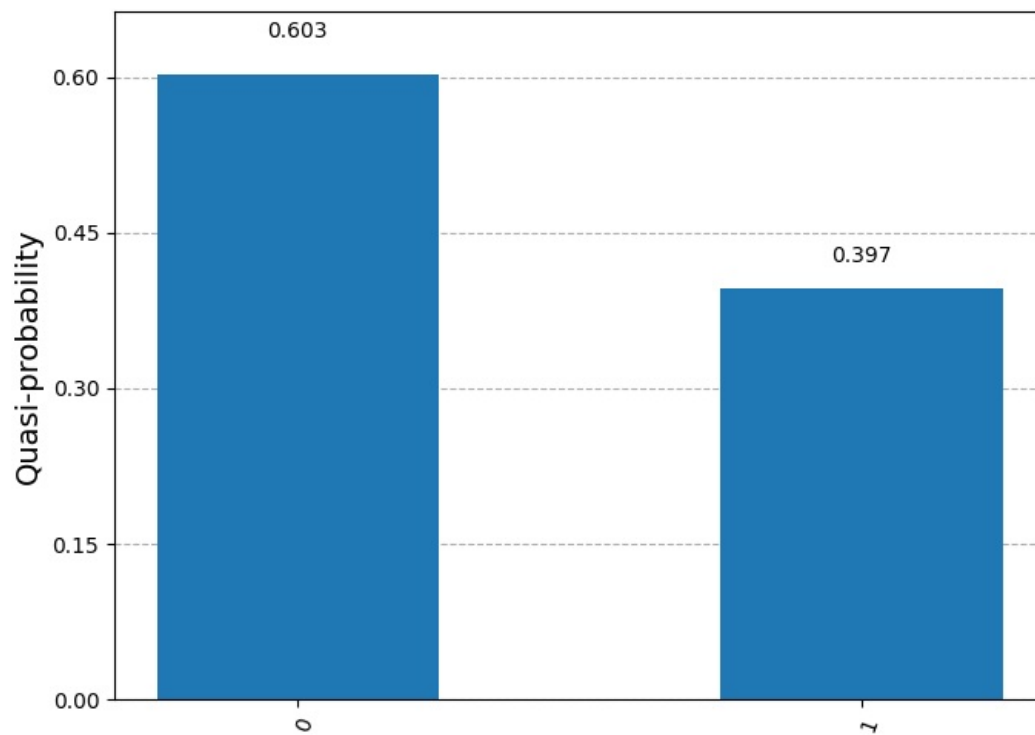
```
In [110.. qc = swap_test_circuit(prepare_psi_00, prepare_phi_minusplus)
qc.draw("mpl")
```


Out[110..



```
In [111.. backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[111..



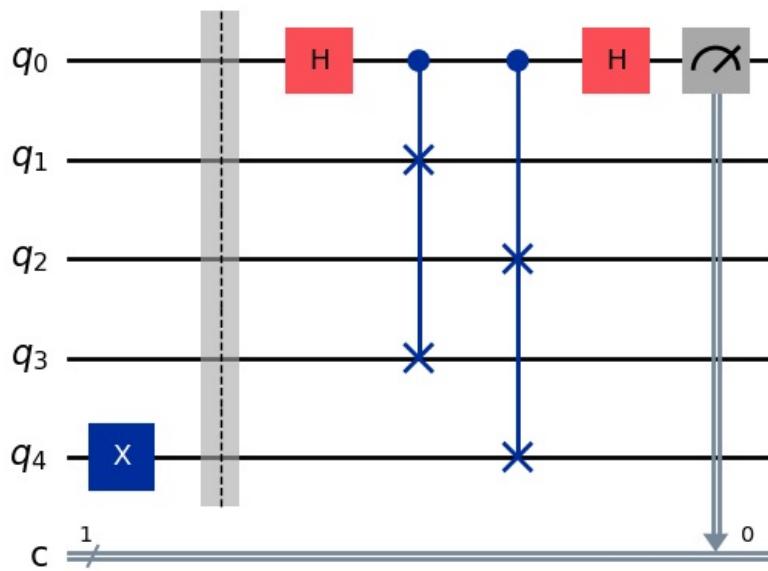
```
In [112.. p0, fid, over = calculate_results(counts)
results["State"].append("-+")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.60303, Fidelity: 0.20605, Overlap: 0.45393

State Φ $|10\rangle$

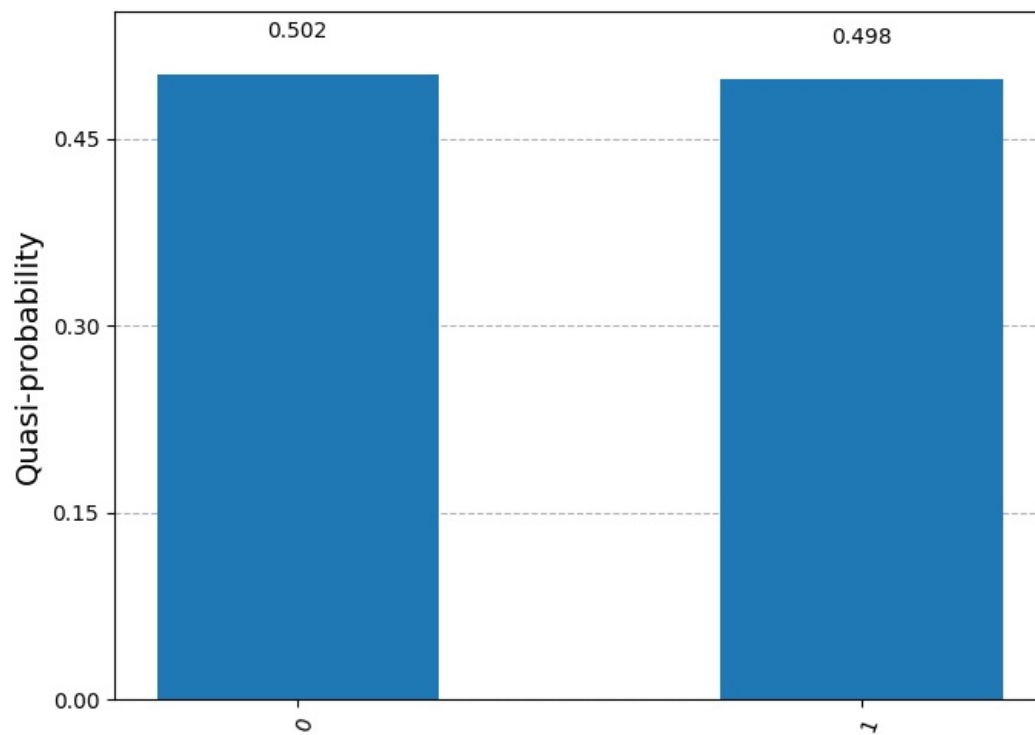
```
In [113.. qc = swap_test_circuit(prepare_psi_00, prepare_phi_10)
qc.draw("mpl")
```

Out[113..



```
In [114.. backend = Aer.get_backend("qasm_simulator")
shots = 2048
transpiled_qc = transpile(qc, backend, optimization_level=3)
job = backend.run(transpiled_qc, shots=shots)
counts = job.result().get_counts()
plot_distribution(counts)
```

Out[114..



```
In [115.. p0, fid, over = calculate_results(counts)
results["State"].append("10")
results["P(0)"].append(p0)
results["Fidelity"].append(fid)
results["Overlap"].append(over)
print(f"P(0): {p0:.5f}, Fidelity: {fid:.5f}, Overlap: {over:.5f}")
```

P(0): 0.50195, Fidelity: 0.00391, Overlap: 0.06250

Task 2 - Results

```
In [117.. df_results = display_lab_results(results)
df_results
```

Out[117...

	State	P(0)	Fidelity	Overlap
0	+0	0.74316	0.48633	0.69737
1	0-	0.74658	0.49316	0.70226
2	-1	0.50342	0.00684	0.08268
3	-0	0.76172	0.52344	0.72349
4	1-	0.61816	0.23633	0.48614
5	--	1.00000	1.00000	1.00000
6	++	0.62646	0.25293	0.50292
7	01	0.47998	-0.04004	0.00000
8	+1	0.50879	0.01758	0.13258
9	1+	0.50439	0.00879	0.09375
10	0+	0.75879	0.51758	0.71943
11	00	1.00000	1.00000	1.00000
12	+-	0.61475	0.22949	0.47905
13	11	0.49609	-0.00781	0.00000
14	-+	0.60303	0.20605	0.45393
15	10	0.50195	0.00391	0.06250

Conclusions

In conclusion, the implementation of the SWAP test successfully enabled the calculation of fidelity and overlap for the assigned quantum patterns, validating the theoretical relationship $F=2P(0)-1$. The results, compiled across 16 different cases, illustrate how specific gate configurations - such as those creating superposition or phase shifts - impact the measurable distance between states. This laboratory exercise highlights the efficiency of quantum state comparison compared to classical methods and serves as a foundational step toward building more complex quantum-enhanced pattern recognition systems and diagnostic tools in quantum information science.